

Claude Multi-Account Fine Tuning

- Claude Toolkit
 - Run many Claude Code accounts on one host — with one unified brain.
 - ✱ Why
 - Features
 - Quick start
 - ✓ Daily commands
 - Documentation
 - Requirements
 - Layout after unification
 - How cross-account sync works
 - Testing
 - Releasing — the LIVE gate is mandatory
 - 🛒 ↩ Rollback
 - License

Claude Toolkit

Run many Claude Code accounts on one host — with one unified brain.

One alias per account. Shared history, memory, plugins, settings, and sessions across all of them. Plus turn any LLM API key into a Claude Code alias, share your whole plugin ecosystem with OpenCode, and auto-name a per-project session on every launch.

version	v1.26.7	platform	Linux macOS	bash	4+ (3.2 auto-reexec)
tests	58 suites green	shellcheck	0 errors	license	see repo

[Quick start](#) · [Features](#) · [Commands](#) · [Guides](#) · [How it works](#)

Why

Running more than one Claude Code account on a machine normally means siloed worlds: each account has its own conversation history, project memory, todos, plans, and installed plugins, and nothing carries over. Claude Toolkit collapses that into **one shared store** — every account is mostly symlinks into it — so switching accounts is just a shell alias, and your work, memory, and plugins follow you. Only the things that *must* stay private (credentials, auth state) remain per-account.

```
claude1      # account 1 – your shared projects, memory, plugins,
              sessions
claude2      # account 2 – the SAME shared projects, memory,
              plugins, sessions
deepseek     # the same ecosystem, running on DeepSeek's best model
```

Features

- **Unified multi-account store.** One alias per account (claude1, claude2, ... or custom names). A single ~/.claude-shared/ holds projects/, todos/, tasks/, plans/, history.jsonl, settings.json, plugins/, CLAUDE.md and more; each account dir is symlinks into it.
- **Cross-account session resume.** The projects/session index in .claude.json is deep-merged across accounts on every launch — a session started under claude1 shows up in claude2's --resume list, and vice versa. Auth keys never cross.
- **Per-project auto-session naming.** Every bare alias launch resumes — or first-time creates — **one long-lived session per project root**, named after the directory in kebab-case (Android 15 → android-15). Open claude2 in a project and you're back in the same ongoing work, properly named — even if the session was previously unnamed. (*Color: a deterministic per-alias /color hint is printed; Claude Code's /color is TUI-only and can't be auto-applied — see [SESSION_COLOR.md](#).*)
- **Any LLM as a Claude Code alias.** claude-providers turns every API key in your keys file into its own alias pointed at that provider's strongest model — fully dynamic (no hardcoded providers/URLs/models), secrets never leave the keys file. [Provider guide](#) →
- **Share your ecosystem with OpenCode.** claude-opencode-sync exposes every Claude plugin's Skills + MCP servers + CLAUDE.md to a host-installed [OpenCode](#) in one command. [OpenCode guide](#) →
- **TOON encoding utility.** toon.mjs / toon_encode.py encode JSON to [TOON](#) (~40% fewer tokens for structured prompt data). [TOON guide](#) →

- ✈ • **Safe & reversible.** Every destructive step is backed up; `claude-rollback` restores everything. Idempotent installers. Verified on Linux and macOS.

Quick start

```
curl -fsSL https://raw.githubusercontent.com/vasic-digital/claude-toolkit/main/scripts/curl-install.sh | bash
```

Clones to `~/claude-toolkit` (or pulls if present), installs missing dependencies via your system package manager, runs the full setup, and wires the aliases into your shell. Re-run anytime to update.

Then open a new shell and:

```
claude-list-accounts    # see what's wired up
claude1                # launch account 1 (auto-resumes this
                        # project's session)
```

Manual install

A. Host already has ≥ 1 `~/claude-*` account:

```
git clone https://github.com/vasic-digital/claude-toolkit.git
    claude-toolkit
cd claude-toolkit
bash scripts/install.sh
    # symlink scripts onto PATH, write managed alias
    # file, source from rc files,
    unify existing
    # accounts, install the TOON dep,
    refresh docs.
exec $SHELL -l
claude-list-accounts
```

B. Clean-slate host (Claude Code installed, no accounts yet):

```
git clone https://github.com/vasic-digital/claude-toolkit.git
    claude-toolkit
cd claude-toolkit
bash scripts/claude-bootstrap.sh --count 2 --yes    # provision
    claude1, claude2
exec $SHELL -l
claude1 /login && claude2 /login                    #
    authenticate each once
```

--aliases personal,work for custom names; --dir-of NAME=PATH to override a config dir. Both installers are **idempotent**.

Daily commands

Command	Purpose
claude-list-accounts	Tabular status: alias, config dir, creds, link health.
claude-add-account	Add an account. Interactive, or --alias NAME --dir PATH --yes.
claude-remove-account --alias NAME	Drop an alias; archive (default) or --delete its dir.
claude-unify	Re-merge state into the shared store (auto-detects ~/.claude-*).
claude-sync-state	Fast jq sync of the .claude.json session index across accounts. Runs automatically around every launch.
claude-session	Per-project session helper (name/id/color/flags) used by the alias wrappers. See SESSION_COLOR.md .
claude-bootstrap	Clean-slate provisioning on a fresh host.
claude-providers	Create/refresh aliases for other LLM providers from your keys file.
claude-opencode-sync	Expose Claude plugin Skills + MCP + CLAUDE.md to OpenCode.
claude-export-docs	Regenerate the long-form guide .html/.pdf from markdown.
claude-rollback	Restore .preunify.* backups and move the shared store aside.

Documentation

Guide	What it covers
Provider Aliases	Turning LLM keys into aliases;
User Guide	transports, overrides, verification.
OpenCode Integration	Sharing Skills + MCP + CLAUDE.md with OpenCode.
Session & Color	Per-project auto-session naming + the per-alias color hint reality.
TOON Integration	Token-efficient JSON encoding utility.
Fine-Tuning deep dive	Full architecture walkthrough (.html/.pdf siblings).
CHANGELOG	Release history.

Requirements

bash, rsync, jq, awk. Optional: node+npm (TOON utility), pandoc (+ weasyprint/wkhtmltopdf/headless chromium) for doc regeneration.

macOS (Homebrew)

```
brew install bash jq rsync gawk pandoc weasyprint
```

macOS ships bash 3.2; the scripts auto-re-exec under Homebrew bash when needed, and only touch ~/.zshrc on Darwin.

Layout after unification

```
~/.claude-shared/                # single source of truth
  projects/ todos/ tasks/ plans/ plugins/ sessions/ ...
  CLAUDE.md  history.jsonl  settings.json  stats-cache.json

~/.claude-<account>/              # per-account – mostly
symlinks
  .credentials.json              # PRIVATE – account-locked
  .claude.json                   # per-account state (non-auth
contents sync)
  mcp-needs-auth-cache.json      # PRIVATE
  projects -> ~/.claude-shared/projects # every shared item is a
symlink

~/.local/share/claude-multi-account/aliases.sh # managed alias
file
```

How cross-account sync works

Claude Code keeps a `projects.<path>` map inside each account's `.claude.json` (`lastSessionId`, MCP status, project memory pointers). Without intervention, account A's projects are invisible to account B even though the JSONL transcripts live in the shared `projects/`. The toolkit fixes this in two layers:

1. **At unify time** — `claude-unify` deep-merges every account's `.claude.json` (rightmost-wins on scalars, recursive union of the `projects` subtree), writing each account's auth-private keys back untouched.
2. **At runtime** — the `cma_run` wrapper calls `claude-sync-state` pull before each launch and push after exit (a fast `jq` merge), so sessions created in one account appear in the others on the next launch.

Auth keys (userID, oauthAccount, firstStartTime, claudeCodeFirstTokenDate) never cross accounts.

Testing

```
bash scripts/tests/run-all.sh          # all hermetic
      suites (sandboxed $HOME)
bash scripts/tests/run-all.sh lib unify session  # subset by
      suffix
bash scripts/tests/run-proof.sh          # hermetic + live
      verifiers + evidence bundle
```

Tests use a sandboxed \$HOME via mktemp — your real ~/.claude* is never touched. The suite is **58 files, all green**, and shellcheck -S error is clean (62 warning-level and 203 style-level suggestions remain); live verifiers (verify*_live.sh) prove behavior against the real OpenCode/provider state and write inspectable evidence to scripts/tests/proof/.

Releasing — the LIVE gate is mandatory

```
claude-release-gate          # sandbox suite + LIVE
      alias smoke
claude-release-gate --verify-providers  # + full LLMsVerifier
      model scan
claude-release-gate --provider poe      # smoke through a
      different provider
```

No release commit without a green claude-release-gate. The sandbox suite proves wrapper *logic* but is structurally blind to real-host state — v1.25.1 shipped fully green while every router alias on the real host was bricked by a PATH-shadowing ccr doppelgänger, and the live smoke then caught two more real defects (a mis-sliced HelixLLM context and ~330k tokens of auto-resumed session history) within minutes of existing. The gate drives the REAL generated alias through the REAL PATH → ccr → route-apply → proxy → backend and asserts the served reply plus the sink-side route. Fail-closed: any layer red = do not release.

↩ Rollback

```
claude-rollback          # == claude-unify --rollback
```

Restores every .preunify.<timestamp> backup and moves the shared store aside to ~/.claude-shared.removed.<timestamp>.

License

See repository.